Partie 3 - Vérification et comparaison VM / Docker

Provenance : mesures de référence de Rémi, conservées pour la partie 3. Voir leur intégration et les différences avec la charge JMeter.

Akrem SOUSSI et Rémi CORTIAL - Refuges - 26 octobre 2026

1. Configuration et protocole

Refuges conserve trois couches : client React servi par Nginx, API FastAPI et MySQL. La validation du 26 octobre 2026 utilise Docker Engine 29.1.3 et Compose 2.40.3 dans Ubuntu 24.04 sous WSL2, sur le même Windows que les VM (8 processeurs logiques, environ 16 Go de RAM). Le projet refuges-validation possède ses propres volumes ; aucune ancienne campagne JMeter ni base de travail n'a été réinitialisée.

Trois répétitions par déploiement, jamais en même temps. Le chronomètre part des conteneurs arrêtés ou des trois VM éteintes, et s'arrête lorsque /api/health confirme aussi la base. Après 8 s de stabilisation, trois échantillons de mémoire sont pris, puis dix appels authentifiés à /api/refuges?limit=20. Les 60 consultations renvoient HTTP 200 avec 4 652 refuges. On retient la médiane des trois essais, chaque essai utilisant la médiane de ses dix latences et de ses trois relevés mémoire.

2. Matrice de joignabilité vérifiée

Depuis / vers	Proxy :80	Métier :8000	MySQL :3306
Windows	Oui, via :8080	Échec attendu observé	Échec attendu observé
Moteur WSL	Oui, via :8080	Échec après filtrage	Échec après filtrage
Proxy	-	Oui	Échec attendu observé
Métier	Oui	-	Oui
Données	Échec attendu observé	Oui	-

Les connexions entre conteneurs ont été testées aux adresses IP réellement attribuées, pour ne pas confondre une absence de résolution DNS avec une isolation réseau. Les tests positifs prouvent que les services cibles étaient en marche. Les états avant et après filtrage sont conservés.

3. Tableau comparatif chiffré

Grandeur	Trois VM	Trois conteneurs
Démarrage, médiane de 3 essais	122,80 s	14,59 s
Mémoire au repos, médiane	3 073,52 MiB (processus VM)	4 671,84 MiB (WSL complet)
Empreinte disque relevée	25,36 GiB (VDI + snapshots)	559,93 MiB (images + volumes)
Latence de consultation HTTP 200	64,80 ms	23,70 ms
Mise en place depuis les sources	Temps historique non chronométré	93,76 s ; moteur/bases d'image présents

La mémoire Docker doit inclure le moteur : la valeur 4 671,84 MiB correspond au processus vmmemWSL complet, pas aux seuls conteneurs. WSL héberge déjà d'autres processus et conserve des caches de construction ; le relevé avant la campagne était de 4 700,02 MiB. Isoler exactement le coût propre du moteur n'est donc pas possible dans cet environnement partagé sans arrêter les autres usages WSL. La RAM des VM est la somme des processus VBoxHeadless/VirtualBoxVM. Aucun gain mémoire net entre architectures n'est déduit de ces deux périmètres.

4. Analyse : les trois questions

De combien les chiffres diffèrent-ils ?

Le démarrage observé passe de 122,80 s à 14,59 s (rapport 8,42). La consultation passe de 64,80 à 23,70 ms (rapport 2,73). Les fichiers VDI, snapshots compris, représentent 25,36 GiB ; la somme des tailles d'images rapportées et des volumes du projet Docker représente 559,93 MiB. Ce dernier chiffre exclut le moteur, ses caches et la distribution WSL ; des couches d'images peuvent être partagées. Ce sont des périmètres explicités, pas une économie physique globale certifiée.

Quelle propriété technique explique chaque écart ?

Les VM démarrent chacune leur système invité, tandis que les conteneurs réutilisent le noyau Linux déjà actif de WSL ; le moteur Docker reste démarré entre les essais. Les trois VM ont chacune 1 vCPU et 2 Go alloués, alors que Compose ne fixe pas de quotas. Les versions des environnements applicatifs et la révision invitée des VM ne sont pas attestées identiques. Ces facteurs empêchent d'attribuer tout l'écart au seul conteneur. Les valeurs décrivent ce poste et ce parcours au repos ; elles ne remplacent pas le protocole commun sous charge de la partie 4.

Que perd le déploiement en conteneurs par rapport aux VM ?

L'isolation des conteneurs est une isolation logicielle par espaces de noms et groupes de contrôle, avec noyau commun. Elle ne fournit pas des noyaux invités indépendants ni la possibilité de faire tourner un autre noyau dans chaque conteneur ; le moteur et le noyau partagé forment une dépendance commune. Sous Windows, WSL ajoute lui-même un environnement Linux. Sur ce moteur natif, l'absence de ports publiés ne bloquait pas les connexions directes depuis WSL aux IP privées. Une chaîne iptables OUTPUT limitée aux réseaux du projet ferme les ports 8000, 3306 et 33060, sans bloquer le proxy ni les flux entre conteneurs.

5. Persistance, peuplement et reproductibilité

Le test de persistance a créé un marqueur dans une table dédiée, recréé les trois conteneurs, puis retrouvé le marqueur et les mêmes compteurs métier. Les identifiants de conteneurs ont changé, les noms des volumes sont restés identiques et la table de test a été retirée. Aucun volume n'a été supprimé. La base contient 4 652 refuges, 156 massifs, 6 communautés et 30 utilisateurs, dont les 25 comptes de charge.

Le Dockerfile métier utilise maintenant metier/requirements.lock, avec les versions exactes de l'image testée. pip check ne signale aucune dépendance cassée. Les exclusions spécifiques des deux Dockerfile sont présentes ; le client utilise npm ci. Les images officielles de base sont fixées par digests.

La reconstruction sans cache des images et l'initialisation d'un volume vide prennent 41,26 s de build puis 52,50 s jusqu'aux services sains, soit 93,76 s. Le moteur et les images de base sont déjà présents : leur installation est exclue. La préparation initiale des VM (systèmes, réseau, services et vérifications) a été réalisée sur plusieurs séances avec l'aide de l'IA, sans chronométrage. Son temps reste non mesuré. Elle nous a paru plus longue, mais aucun gain chiffré n'en est déduit : les périmètres ne sont pas comparables. Ces mesures ne remplacent pas rétroactivement les 45 s et 2 710 Mo historiques de l'étape 10.

6. Preuves

• Versions, sorties CLI, matrices et mesures brutes.

• Procédure complète de construction et démarrage.

• Relevé visuel des trois services : rendu des sorties CLI archivées, sans reconstitution d'une interface Docker Desktop.